aws + ROS

Deploy ROS Noetic on AWS Cloud

Complete Step-by-Step Solution Guide

Company	Role	Platform	ROS Version
IS Robotics	Software Engineer	AWS EC2	Noetic Ninjemys

Ubuntu 20.04 LTS Operating System	t2.micro General Purpose EC2	30 GB EBS Storage	Free Tier AWS Account

■ Project Overview

This guide provides a complete, end-to-end solution for deploying **ROS Noetic Ninjemys** on **Amazon Web Services (AWS)**. Once deployed, developers at IS Robotics can remotely access the cloud instance to run robotic simulations, monitor nodes, and collaborate on ROS-based projects without needing powerful local hardware.

■ High-Level Architecture

Component	Service / Tool	Specification	Purpose
Cloud Provider	Amazon Web Services	ap-south-1 (Mumbai)	Hosting the VM
Compute	EC2 Instance	t2.micro (1 vCPU, 1 GB RAM)	Run ROS processes
OS Image	Ubuntu 20.04 LTS AMI	Official Canonical AMI	ROS Noetic compatibility
Storage	EBS Volume	30 GB gp2/gp3	OS + ROS packages
Network	Security Group	SSH (22), Custom TCP (11311)	Access & ROS Master
ROS	ROS Noetic Desktop-Full	Latest stable release	Simulation & development
Access	SSH via Terminal / VS Code	Key-pair authentication	Developer access

■ **NOTE:** The t2.micro instance is available under AWS Free Tier (750 hrs/month for 12 months). For heavier simulations with Gazebo/RViz, consider upgrading to t3.medium or t3.large.

■ Part 1 — Setting Up AWS EC2 Instance

Step 1 — Create an AWS Account

If you don't have an AWS account, visit **https://aws.amazon.com** and click **Create a Free Account**. You will need:

- A valid email address
- Credit/debit card (no charges under Free Tier)
- Phone number for verification

✔ **TIP:** Select the **Free Tier** plan. You get 750 hours/month of t2.micro usage free for the first 12 months.

Step 2 — Launch an EC2 Instance

Open EC2 Dashboard	Log in to AWS Console → Search 'EC2' → Click **Launch Instance**.
Name Your Instance	Enter a meaningful name, e.g., **ros-noetic-server**.
Choose AMI	Search for **Ubuntu Server 20.04 LTS** → Select the 64-bit (x86) version. Ensure it shows **Free tier eligible**.
Choose Instance Type	Select **t2.micro** (1 vCPU, 1 GiB RAM) — Free Tier eligible.
Create Key Pair	Click **Create new key pair** → Name it (e.g., ros-key) → Choose RSA / .pem → Download it. **Keep this file safe — you cannot re-download it!**
Configure Storage	Set root volume to **30 GB** (gp2 or gp3). Leave other settings as default.
Security Group	Add inbound rules: SSH (port 22) from your IP, and Custom TCP port **11311** for ROS Master communication.
Launch	Review settings → Click **Launch Instance**. Wait ~2 minutes for the instance to start.

Security Group Inbound Rules

Type	Protocol	Port Range	Source	Description
SSH	TCP	22	Your IP / 0.0.0.0/0	Remote terminal access
Custom TCP	TCP	11311	0.0.0.0/0	ROS Master default port
Custom TCP	TCP	11312	0.0.0.0/0	ROS parameter server

■ Part 2 — Connect to Your EC2 Instance via SSH

Once the instance is running, connect from your local machine using SSH.

Step 3 — Set File Permissions on Key

On Linux/Mac terminal:

```
chmod 400 ~/Downloads/ros-key.pem
```

Step 4 — SSH into the Instance

Get your **Public IPv4 DNS** from the EC2 console (Instance Details panel).

```
ssh -i ~/Downloads/ros-key.pem ubuntu@



# Example:

ssh -i ~/Downloads/ros-key.pem
ubuntu@ec2-13-232-45-67.ap-south-1.compute.amazonaws.com
```

✔ **TIP:** Type **yes** when prompted about the host fingerprint. You should see the Ubuntu welcome banner once connected.

■ Part 3 — Install ROS Noetic on Ubuntu 20.04

Run all the following commands on your **EC2 terminal** (after SSH login).

Step 5 — Update System Packages

```
sudo apt update && sudo apt upgrade -y
```

Step 6 — Configure Ubuntu Repositories

Enable the **universe**, **restricted**, and **multiverse** repos:

```
sudo add-apt-repository universe

sudo add-apt-repository restricted

sudo add-apt-repository multiverse
```

```
sudo apt update
```

Step 7 — Add ROS Package Repository

Configure your system to accept software from **packages.ros.org**:

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu \

$(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

Step 8 — Set Up Authentication Keys

```
sudo apt install curl -y


curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc \

| sudo apt-key add -
```

Step 9 — Update Package Index

```
sudo apt update
```

■ ROS Installation Variants

Choose the installation variant that best suits your needs:

Variant	Command	Size	Includes	Recommended For
Desktop-Full ★ Recommended	sudo apt install ros-noetic-desktop-full	~2.5 GB	ROS core + RViz + Gazebo + perception packages	Robotic simulation & full development
Desktop	sudo apt install ros-noetic-desktop	~1.5 GB	ROS core + RViz (no Gazebo)	Development without 3D simulation
ROS-Base	sudo apt install ros-noetic-ros-base	~200 MB	Bare minimum ROS packages only	Headless servers & CI/CD pipelines

Step 10 — Install ROS Noetic Desktop-Full (Recommended)

```
sudo apt install ros-noetic-desktop-full -y
```

This may take 5–15 minutes depending on network speed.

■ Environment Setup

Step 11 — Source the ROS Setup Script

You must source the ROS setup script in **every bash terminal** before using ROS commands:

```
source /opt/ros/noetic/setup.bash
```

Step 12 — Auto-Source on Every Login (Recommended)

Add the source command to **~/.bashrc** so it loads automatically:

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc

source ~/.bashrc
```

Step 13 — Install rosinstall & Build Tools

```
sudo apt install python3-rosdep python3-rosinstall \

python3-rosinstall-generator python3-wstool build-essential -y
```

```
sudo rosdep init

rosdep update
```

■ **INFO:** If rosdep init says 'file already exists', skip it and run rosdep update only.

■ Part 4 — Verify the Installation

Step 14 — Check ROS Version

```
rosversion -d

# Expected output: noetic
```

Step 15 — Start ROS Master

Open a terminal and run:

```
roscore
```

You should see output ending with: ***started core service [/rosout]***

Step 16 — Run a Test Node (talker/listener)

Open a second SSH terminal and run:

```
# Terminal 2 — run publisher

rosrun roscpp_tutorials talker


# Terminal 3 — run subscriber

rosrun roscpp_tutorials listener
```

✔ **TIP:** If you see messages flowing between talker and listener, your ROS installation is working correctly! ■

■ Next Steps & Best Practices

■ Create a ROS Workspace

```
mkdir -p ~/catkin_ws/src && cd ~/catkin_ws && catkin_make

echo "source ~/catkin_ws/devel/setup.bash" >> ~/.bashrc
```

■ Install Gazebo Simulator

```
sudo apt install gazebo11 ros-noetic-gazebo-ros-pkgs \
```

```
ros-noetic-gazebo-ros-control -y
```

■ Remote GUI with X11 Forwarding (RViz/Gazebo)

```
# Connect with X11 forwarding enabled:

ssh -X -i ros-key.pem ubuntu@


# Then launch RViz:

rosrun rviz rviz
```

■ Stop Instance to Save Costs

```
# From AWS Console → EC2 → Select Instance → Instance State → Stop

# (Your data is preserved; you only pay for storage when stopped)
```

Troubleshooting Common Issues

Issue	Likely Cause	Solution
SSH: Permission denied	Wrong key or permissions	Run: chmod 400 ros-key.pem
SSH: Connection timed out	Security group missing SSH rule	Add inbound rule: TCP 22 from your IP
apt-key deprecated warning	Ubuntu 22+ key format	Use signed-by method (see ROS wiki)
roscore not found	ROS not sourced	Run: source /opt/ros/noetic/setup.bash
rosdep init: file exists	Already initialized	Skip init, just run: rosdep update
Gazebo crashes / slow	t2.micro low RAM	Stop Gazebo GUI, use headless mode or upgrade instance
Out of disk space	30GB full	Clean: sudo apt autoremove && sudo apt clean

Quick Reference Cheatsheet

Task	Command
Source ROS environment	`source /opt/ros/noetic/setup.bash`
Start ROS Master	`roscore`
List active nodes	`rosnode list`
List active topics	`rostopic list`
Echo a topic	`rostopic echo /topic_name`
Check node info	`rosnode info /node_name`
Kill ROS master	`Ctrl+C in roscore terminal`
Build catkin workspace	`cd ~/catkin_ws && catkin_make`
Launch a .launch file	`roslaunch package_name file.launch`
Install a ROS package	`sudo apt install ros-noetic-PACKAGE`
Check ROS distro	`rosversion -d`

■ Mission Complete!

You have successfully deployed **ROS Noetic** on an **AWS EC2** instance. Developers at IS Robotics can now SSH into the cloud server, launch ROS nodes, run simulations with Gazebo, visualize data with RViz, and collaborate on robotic projects — all from the cloud, from anywhere in the world.